HEALTHTRACK+ FOR WOMEN: TECHNICAL WORKING GUIDE

Complete Internal Architecture: React, TypeScript, Kotlin, Capacitor, AI & Doctor Portal

Presenter & Viva Defense Reference Document | Clean Monochrome Edition

MODULE 1: The Big Picture (How TypeScript, Kotlin & Capacitor Connect)

1. TypeScript & React 19 (User Interface Layer)

• Location: src/ directory (App.tsx, components/, utils/).
• Role: Controls all screens, calendars, charts, forms, symptom buttons, and animations.
• Execution: Compiled by Vite into optimized JavaScript/HTML that renders inside the mobile app view.

2. Capacitor Bridge (The Link between Web and Phone)

• Role: Wraps the React web code inside a real native Android APK (com.aura.health).
• How it works: Creates a two-way communication bridge between JavaScript and native Java/Kotlin code.
• Trigger: When TypeScript needs phone hardware (camera for strip scanner, ONNX AI chip, hardware back button), it calls Capacitor plugins across this bridge.

3. Kotlin / Java (Native Android Engine)

• Location: android/app/src/main/java/com/aura/health/OnnxPredictorPlugin.java.
• Role: Interacts directly with the Android mobile operating system and hardware.
• Execution: Loads pcos_rotterdam_v4.onnx directly into phone RAM and runs Microsoft ONNX Runtime C++ engine for sub-30ms offline AI inference.

MODULE 2: Button Click to Database Storage (Step-by-Step Flow)

Detailed Execution Flow When User Taps a Button (e.g. +250ml Water or Period Log)

• Step 1 (UI Click): User taps button -> onClick triggers in React component (QuickLogBar.tsx / WaterTracker.tsx).
• Step 2 (Local State Update): Component calls setLogs() -> React state updates instantly with 0ms visual lag.
• Step 3 (Repository Dispatch): Calls getDatabaseProvider().saveDailyLog(uid, date, logData) via abstract repository interface.
• Step 4 (On-Device Storage): Log is immediately saved to phone storage (localStorage) so data is never lost even if phone reboots or loses connection.
• Step 5 (Firestore Cloud Sync): Firebase SDK asynchronously sends JSON document to users/{uid}/dailyLogs/{YYYY-MM-DD}.
• Step 6 (Offline Handling): If offline, Firebase automatically queues data in local cache and uploads automatically when internet reconnects.

MODULE 3: How the AI Chatbot Works Internally

3-Tier Failover Architecture in GeminiChatbot.tsx

• Context Injection: Before query is sent, the system secretly appends live user status: Cycle Day, Phase, and current symptoms.
• Tier 1 (Custom OpenRouter Key): If user enters their own key in Settings, calls models like Llama 3.3 or DeepSeek Chat.
• Tier 2 (Real-Time Live Engine): Free 24/7 AI fallback (Pollinations AI) ensuring user always gets a live response without requiring an API key.
• Tier 3 (Offline Clinical Heuristics): If completely disconnected (airplane mode), src/utils/aiMasterEngine.ts matches keywords across 20+ clinical topics, Rotterdam guidelines, and Ayurvedic remedies. It never shows a connection error.

MODULE 4: How On-Device ML & ONNX Work on Android

Exact Journey of the 19-Parameter Diagnostic Vector

• Step 1 (Form Input): User fills 19 Rotterdam biomarkers (Age, Menarche, BMI, WHR, Cycle Length, Variance, Hirsutism score, Acne, etc.).
• Step 2 (Vector Assembly): buildPcosRotterdamVector() formats input into exact 19-element Float32 array.
• Clinical Rule 1: Missing lab values (LH/FSH, Testosterone, Insulin, TSH) are set to -1.0 per decision tree protocol.
• Clinical Rule 2: Active birth control sets mean cycle to 28.0d and variance to 0.5 to prevent symptom masking.
• Step 3 (Bridge Call): TypeScript executes OnnxPredictor.runInference({ data: vector }) over the Capacitor bridge.
• Step 4 (Native Execution): OnnxPredictorPlugin.java feeds vector into OnnxTensor and runs session.run() in C++ runtime.
• Step 5 (Rotterdam Output): Model outputs 5 probabilities: Class 0 (Baseline), Class 1 (Phenotype A), Class 2 (Phenotype B), Class 3 (Phenotype C), Class 4 (Phenotype D).
• Speed & Privacy: Entire prediction finishes in under 30 milliseconds with 100% data privacy (zero cloud transmission).

MODULE 5: How Menstrual Cycle & Fertile Window Math Works

Mathematical Formulas in src/utils/cycleCalculations.ts

• Days Passed: (Today Date - Last Period Start Date) / 86,400,000 milliseconds.
• Current Cycle Day: ((Days Passed % Cycle Length) + Cycle Length) % Cycle Length + 1.
• Ovulation Day: Cycle Length - Luteal Phase Length (Default 14 days). E.g., 28 - 14 = Day 14.
• Fertile Window: Ovulation Day minus 5 days through Ovulation Day plus 1 day (7-day window based on sperm longevity).
• Phase Probability: Menstrual Phase (3% conception), Follicular Phase (8%), Fertile Window (18%-28%), Ovulation Peak (33%), Luteal Phase (4%).

MODULE 6: How Doctor Portal Connects to Patient Data

Clinical Data Pipeline Between Patient Mobile App & Doctor Web Portal

• Step 1: Patient logs multi-cycle data and runs Rotterdam AI on mobile app.
• Step 2: App packages records into standardized summary synced to Firestore collection users/{uid}/medicalSummary.
• Step 3: Doctor logs into responsive Web Portal using verified clinician account (Role-Based Access Control).
• Step 4: Doctor dashboard renders patient cycle regularity graph, BBT biphasic curve, and Rotterdam Phenotype risk card.
• Step 5: Doctor writes digital consultation advice / lifestyle prescription which syncs back to patient app in real time.

MODULE 7: Viva Voce "Cheat Sheet" – Quick Responses for Examiners

Top 4 Questions & Confident Answers

• Q1: What does Kotlin do if the app is written in TypeScript? -> "TypeScript manages the UI and screens. Kotlin acts as the native Android bridge that loads the ONNX neural model into memory and executes it using mobile hardware acceleration."
• Q2: How is data stored? -> "We use an Abstract Repository Pattern with dual providers: LocalStorage for instant zero-lag offline access, and Google Cloud Firestore for secure cloud backup."
• Q3: Why run ML on-device instead of on a server? -> "Reproductive health data is highly sensitive. On-device ONNX execution ensures 100% data privacy (data never leaves the phone) and provides instant sub-30ms results without needing internet."
• Q4: How does the Doctor Portal work? -> "It is a web portal that reads patient-authorized clinical summaries from Firestore, displaying cycle trends, Rotterdam Phenotype A-D diagnostic classifications, and allowing physicians to issue consultation notes."